Introduction to Capsule Networks [Code Tutorial]

Kisore Senthilkumar
Dharaneeshwar Shrisai Kumaraguru

Brief Overview:

A capsule is a group of neurons whose activity vector represents the instantiation parameters of a specific type of entity such as an object or an object part. Capsule networks are an extension of CNNs. They were made to address the limitations of traditional CNNs and unlike CNNs they rely on pooling layers that lose spatial hierarchies. In this paper we will look further into how Capsule Networks work and learn how to implement them with code in PyTorch.

Capsule Networks Explained:

Capsule networks are mainly made of capsules that are a bunch of neurons that are used to encode both the presence and pose of features. With the spatial hierarchy maintenance between the features they get close to natural human perception and pick up context clues better than a typical NN. They use an iterative routing algorithm called dynamic routing to determine how much information a lower-level capsule should send to a higher-level capsule instead of max pooling. CapsNets also use a squashing function to maintain an output between 0 and 1 to indicate presence of a feature or not. CapsNets are better at recognizing objects under different transformations, require fewer training data while reducing the need for data augmentation. Despite these advantages, CapsNets are computationally expensive and difficult to scale, which has limited their widespread adoption compared to CNNs. Optimizing their efficiency remains an area of active research.

Dataset Used:

The CIFAR-10 dataset (Canadian Institute for Advanced Research) is a widely used benchmark dataset in computer vision for training and evaluating machine learning models, especially convolutional neural networks (CNNs). It consists of 60,000 images, each being a 32x32 pixel image. The images are divided into 10 classes and each class has 6,000 images. They include images from different angles, lighting conditions and backgrounds. This is the optimal dataset for our case as we would be able to demonstrate CapsNets capabilities in these conditions. The cifar-10 and cifar-100 datasets can be found at https://www.cs.toronto.edu/~kriz/cifar.html . Here's how you can load the dataset for usage.

```python
transform = transforms.Compose([
                transforms.ToTensor(),
                transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

P_transform = transforms.Compose([
    transforms.ToTensor()
])

preview = datasets.CIFAR10(root='./data', train=False, download=True, transform=P_transform)
class_names = preview.classes

# Load CIFAR-10
dataset = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
```

Model Architecture:

Initial Convolutional Layer:

This layer serves as the feature extractor, transforming the input image into a set of low-level feature maps. Unlike traditional CNNs, CapsNets do not use pooling layers because pooling loses spatial relationships.

```python
class InitialConv(nn.Module):
    def __init__(self, in_channels=3, out_channels=256, kernel_size=9):
        super(InitialConv, self).__init__()
        self.conv_layer = nn.Conv2d(in_channels, out_channels, kernel_size, stride=1)

    def forward(self, x):
        return F.relu(self.conv_layer(x))
```

It is a single convolutional layer with 3 input channels for RGB image (for grayscale images this would default to 1) and has 256 output channels. It has a kernel size of 9x9 and a stride of 1 to ensure that there is minimal reduction in spatial size. We use the ReLU activation function in this layer to introduce non-linearity.

How this layer works is that it takes an image and applies 256 convolutional filters of size 9x9 and outputs 256 feature maps. This layer captures local patterns like edges and textures. The large kernel size ensures that it learns complex features.

Primary Capsule:

This layer converts scalar feature maps into vectorized capsule representations. Each capsule represents a local part-whole relationship.

```
class PrimaryCapsules(nn.Module):
    def __init__(self, capsule_dim=8, in_channels=256, capsule_channels=32, kernel_size=9, num_routes=32 * 6 * 6):
        super(PrimaryCapsules, self).__init__()
        self.num_routes = num_routes
        self.capsule_layers = nn.ModuleList([
            nn.Conv2d(in_channels, capsule_channels, kernel_size=kernel_size, stride=2, padding=0)
            for _ in range(capsule_dim)])

    def forward(self, x):
        u = [layer(x) for layer in self.capsule_layers]
        u = torch.stack(u, dim=1)
        u = u.view(x.size(0), self.num_routes, -1)
        return self.squash(u)

    def squash(self, tensor):
        norm = (tensor ** 2).sum(-1, keepdim=True)
        return (norm * tensor) / ((1.0 + norm) * torch.sqrt(norm + 1e-8))
```

The structure consists of 32 capsule channels each generating an 8D vector output. Uses the same 9x9 conv kernels with a stride of 2 to reduce spatial dimensions. This is where unlike CNNs we don't use max pooling to retain the spatial relationships. We introduce something called a squashing function to normalize capsule magnitudes between 0 and 1. The input feature maps are passed through 32 convolutional capsule channels and each channel outputs the feature maps. These are reshaped into 2048 capsule vectors each of 8 dimensions. The squashing function ensures that the output is between 0 and 1 and this preserves the relative importance of features. It is important to note that the capsules output vectors and not scalars. This also encodes the position, scale and orientation of the detected features.

Squashing Function:

$$v_j = \frac{||s_j||^2}{1 + ||s_j||^2} \frac{s_j}{||s_j||}$$

This nonlinearity forces shorter vectors to shrink towards 0 and longer vectors to approach 1. We do this so we can make sure the magnitude of the capsule indicates the presence of a feature as mentioned earlier. Magnitude being closer 1 means it's more confident that the feature is present.

Digit Capsules:

Transforms the primary capsules into digit capsules, where each capsule represents a class (0-9). Uses dynamic routing to determine which primary capsules contribute to each digit capsule. Each Digit Capsule is a 16-dimensional vector (compared to the 8D vectors in primary capsules) and there are 10 capsules, one for each class. The model uses a trainable transformation matrix (W) to understand how small parts connect to form whole objects. It applies three rounds of dynamic routing to refine the results.

```
class DigitCapsules(nn.Module):
    def __init__(self, num_classes=10, num_routes=32 * 6 * 6, in_dim=8, out_dim=16):
        super(DigitCapsules, self).__init__()
        self.W = nn.Parameter(torch.randn(1, num_routes, num_classes, out_dim, in_dim))
        self.num_classes = num_classes
        self.num_routes = num_routes

    def forward(self, x):
        batch_size = x.size(0)
        x = torch.stack([x] * self.num_classes, dim=2).unsqueeze(4)

        W = torch.cat([self.W] * batch_size, dim=0)
        u_hat = torch.matmul(W, x)

        b_ij = Variable(torch.zeros(1, self.num_routes, self.num_classes, 1)).cuda()

        num_iterations = 3
        for iteration in range(num_iterations):
            c_ij = F.softmax(b_ij, dim=1)
            c_ij = torch.cat([c_ij] * batch_size, dim=0).unsqueeze(4)

            s_j = (c_ij * u_hat).sum(dim=1, keepdim=True)
            v_j = self.squash(s_j)

            if iteration < num_iterations - 1:
                a_ij = torch.matmul(u_hat.transpose(3, 4), torch.cat([v_j] * self.num_routes, dim=1))
                b_ij = b_ij + a_ij.squeeze(4).mean(dim=0, keepdim=True)

        return v_j.squeeze(1)

    def squash(self, tensor):
        norm = (tensor ** 2).sum(-1, keepdim=True)
        return (norm * tensor) / ((1.0 + norm) * torch.sqrt(norm + 1e-8))
```

How this works is that each primary capsule is transformed using a learned weight matrix W into a 16D prediction vector

$$\hat{u}_j = W_{ij} u_i$$

Each transformed vector votes for the respective digit capsule. The routing coefficients c_ij are determined using Softmax and this ensures that the sum of votes for each primary capsule is 1. The weighted sum of cotes s_j determines the final capsule output. The output is then squashed just like it was done in the primary capsules. If a lower capsule is contributing strongly to a class, it is reinforced through routing iterations. We need to remember that this dynamic routing is what sets apart the model from traditional CNNs to preserve spatial relationships.

Capsule Decoder:

This helps with regularization by reconstructing the input image and ensures that capsules learn meaningful feature representations. It is a fully connected network with 3 layers where the first layer consists of 512 neurons with ReLU, second layer consists of 1024 with ReLU and the last layer outputs an image of 32x32x3 pixels with Sigmoid activation function.

```
class CapsuleDecoder(nn.Module):
    def __init__(self, img_width=28, img_height=28, img_channels=1):
        super(CapsuleDecoder, self).__init__()
        self.reconstruction = nn.Sequential(
            nn.Linear(16 * 10, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, 1024),
            nn.ReLU(inplace=True),
            nn.Linear(1024, img_width * img_height * img_channels),
            nn.Sigmoid()
        )

    def forward(self, digit_caps, input_img):
        class_probs = torch.sqrt((digit_caps ** 2).sum(2))
        class_probs = F.softmax(class_probs, dim=0)

        _, max_indices = class_probs.max(dim=1)
        mask = torch.eye(10, device=digit_caps.device)  # This puts it on the same device
        mask = mask.index_select(dim=0, index=max_indices.view(-1))
        masked_caps = (digit_caps * mask[:, :, None, None]).view(digit_caps.size(0), -1)
        recon = self.reconstruction(masked_caps)
        recon = recon.view(-1, input_img.size(1), input_img.size(2), input_img.size(3))
        return recon, mask
```

The class probability is calculated by computing the L2 norms of each digit capsule and applying softmax to them. The masking mechanism extracts the most active capsule and sets all other capsules to 0 before decoding. The reconstruction segment passes the masked capsule through three fully connected layers and the output is reshaped

into a 32x32x3 image. This layer acts as a regularization mechanism to prevent overfitting and forces capsules to retain important object details.

The Loss Functions:

This model uses both the margin loss and reconstruction loss functions.

Margin Loss Function:

Ensures correct class capsules have high magnitudes and incorrect ones have low magnitudes.

$$\sum_k T_k \cdot \max(0, m^+ - ||v_k||)^2 + \lambda \cdot (1 - T_k) \cdot \max(0, ||v_k|| - m^-)^2$$

In our case

- T_k value is 1 if correct else 0
- Our m^+ and m^- values are 0.9 and 0.1
- Our factor value is 0.5

```
def margin_loss(self, x, labels):
    batch_size = x.size(0)
    lengths = torch.sqrt((x ** 2).sum(dim=2, keepdim=True))
    left = F.relu(0.9 - lengths).view(batch_size, -1)
    right = F.relu(lengths - 0.1).view(batch_size, -1)
    loss = labels * left + 0.5 * (1.0 - labels) * right
    return loss.sum(dim=1).mean()
```

Reconstruction Loss Function:

This loss function uses MSE to compare original image and reconstructed image

$$\text{Reconstruction Loss} = \text{MSE}(X, \hat{X})$$

Where X is the original image and the other input is the reconstructed image. This ensures both classification accuracy and meaningful feature representation.

```
def reconstruction_loss(self, original, reconstruction):
    return self.mse(reconstruction.view(reconstruction.size(0), -1), original.view(reconstruction.size(0), -1)) * 0.0005
```

Full Model Flow:

```python
class CapsuleNetwork(nn.Module):
    def __init__(self, config=None):
        super(CapsuleNetwork, self).__init__()
        self.conv = InitialConv(3,256,9)
        self.primary_caps = PrimaryCapsules(8, 256, 32, 9, 32*8*8)
        self.digit_caps = DigitCapsules(10, 32*8*8, 8, 16)
        self.decoder = CapsuleDecoder(32, 32, 3)
        self.mse = nn.MSELoss()

    def forward(self, x):
        features = self.conv(x)
        primary_output = self.primary_caps(features)
        digit_output = self.digit_caps(primary_output)
        recon, mask = self.decoder(digit_output, x)
        return digit_output, recon, mask

    def loss(self, original, digit_output, labels, reconstruction):
        return self.margin_loss(digit_output, labels) + self.reconstruction_loss(original, reconstruction)

    def margin_loss(self, x, labels):
        batch_size = x.size(0)
        lengths = torch.sqrt((x ** 2).sum(dim=2, keepdim=True))
        left = F.relu(0.9 - lengths).view(batch_size, -1)
        right = F.relu(lengths - 0.1).view(batch_size, -1)
        loss = labels * left + 0.5 * (1.0 - labels) * right
        return loss.sum(dim=1).mean()

    def reconstruction_loss(self, original, reconstruction):
        return self.mse(reconstruction.view(reconstruction.size(0), -1), original.view(reconstruction.size(0), -1)) * 0.0005
```

The model takes in an input image of dimensions 32x32x3 sends it through the initial convolutional layer, then it is sent to the primary capsules for normalization. After that it is sent to the digit capsules for feature presence detection. Then it is sent to the decoder where it reconstructs the image. The loss functions are used for training.

Training Process:

Training Phase:

1. A forward pass is done with tracking variables initialized. The gradients are cleared before each forward pass to make sure they don't contribute to the current batch's calculations. We convert the labels to one-hot encoding for capsule network compatibility. During the forward pass, the input image is passed through all layers of the model and we get the output of the classifications and the reconstructed images for comparison.

2. Using the margin and reconstruction loss functions we compute the total capsule network loss.

3. Depending on the loss the gradients are computed and weights are modified using the optimizer.step(). This is the backward pass step.

4. The training accuracy is computed by comparing masked.argmax(dim=1) with labels.argmax(dim=1).
5. Update the tracking variables

Validation Phase:

1. Set model to evaluation mode (model.eval()) to disable gradient updates.

2. Iterate over validation mini-batches:

3. Perform forward pass without updating gradients.

4. Compute validation loss.

5. Compute validation accuracy.

6. Store average validation loss and accuracy.

7. Update best validation accuracy and check for early stopping.

At the end of the training print epoch time, losses and accuracies.

```
def train_and_validate(
    model: torch.nn.Module,
    train_loader: torch.utils.data.DataLoader,
    val_loader: torch.utils.data.DataLoader,
    device: torch.device,
    num_epochs: int,
    optimizer: torch.optim.Optimizer,
    early_stopping: int = None,
):
    train_losses, train_accuracies = [], []
    validation_losses, validation_accuracies = [], []
    epoch_times = []
    best_val_accuracy = 0.0
    epochs_without_improvement, train_predicted_true = 0,0
    start_time = time.time()

    for epoch in range(num_epochs):
        epoch_start = time.time()
        model.train()
```

```python
train_loss, train_predicted_true, correct_train, total_train = 0.0, 0, 0, 0

for inputs, labels in train_loader:
    one_hot_targets = F.one_hot(labels, num_classes=10).float().to(device)
    inputs, one_hot_targets = inputs.to(device), one_hot_targets.to(device)

    optimizer.zero_grad()
    outputs, reconstructions, masked = model(inputs)
    loss = model.loss(inputs, outputs, one_hot_targets, reconstructions)
    loss.backward()
    optimizer.step()

    train_loss += loss.item()
    predicted = masked.argmax(dim=1)
    train_predicted_true += predicted.eq(one_hot_targets.argmax(dim=1)).sum().item()
    total_train += labels.size(0)

train_losses.append(train_loss / len(train_loader))
train_accuracies.append(train_predicted_true / total_train)

model.eval()
val_loss, correct_val, total_val = 0.0, 0, 0
```

```python
for inputs, labels in val_loader:
    one_hot_targets = F.one_hot(labels, num_classes=10).float()
    inputs, one_hot_targets = inputs.to(device), one_hot_targets.to(device)

    optimizer.zero_grad()
    outputs, reconstructions, masked = model(inputs)
    loss = model.loss(inputs, outputs, one_hot_targets, reconstructions)
    loss.backward()
    optimizer.step()

    val_loss += loss.item()
    predicted = masked.argmax(dim=1)
    correct_val += predicted.eq(one_hot_targets.argmax(dim=1)).sum().item()
    total_val += labels.size(0)

val_loss_epoch = val_loss / len(val_loader)
val_acc_epoch = correct_val / total_val
validation_losses.append(val_loss_epoch)
validation_accuracies.append(val_acc_epoch)
```

```python
        if val_acc_epoch > best_val_accuracy:
            best_val_accuracy = val_acc_epoch
            epochs_without_improvement = 0
        else:
            epochs_without_improvement += 1

        if early_stopping and epochs_without_improvement >= early_stopping:
            print(f"Early stopping triggered at epoch {epoch+1}")
            break

        epoch_time = time.time() - epoch_start
        epoch_times.append(epoch_time)

        print(f"Epoch {epoch+1}/{num_epochs} - Time: {epoch_time:.2f} sec")
        print(f"Training Loss: {train_losses[-1]:.4f}, Training Accuracy: {train_accuracies[-1]:.4f}")
        print(f"Validation Loss: {validation_losses[-1]:.4f},Validation Accuracy: {validation_accuracies[-1]:.4f}")

    total_time = time.time() - start_time
    avg_epoch_time = sum(epoch_times) / len(epoch_times)

    print(f"\nTotal Training Time: {total_time:.2f} seconds ({total_time/60:.2f} minutes)")
    print(f"Average Time per Epoch: {avg_epoch_time:.2f} seconds")

    return train_losses, train_accuracies, validation_losses, validation_accuracies
```

Testing Process:

1. Set the model to evaluation mode (model.eval()).

2. Track the following:

   labels_test_total → Ground truth labels.
   test_predictions → Model's predicted labels.
   test_probabilities → Model's class probability outputs.
   test_loss → Total test loss.
   test_accuracy → Final accuracy.

3. Iterate through the test dataset:

4. Perform forward pass.

5. Compute test loss (similar to validation loss).

6. Store predictions & ground truth labels.

7. Compute overall test accuracy.

```python
def test_model(model: torch.nn.Module, dataloader: torch.utils.data.DataLoader, device: torch.device) -> tuple:
    model.eval()
    test_loss, correct_test = 0.0, 0
    test_predictions, labels_test_total, test_probabilities = [], [], []

    for inputs, labels in dataloader:
            one_hot_targets = F.one_hot(labels, num_classes=10).float()
            inputs, one_hot_targets = inputs.to(device), one_hot_targets.to(device)

            optimizer.zero_grad()
            outputs, reconstructions, masked = model(inputs)
            loss = model.loss(inputs, outputs, one_hot_targets, reconstructions)
            loss.backward()
            optimizer.step()

            test_loss += loss.item()
            probs = masked
            predicted = masked.argmax(dim=1)

            correct_test += predicted.eq(one_hot_targets.argmax(dim=1)).sum().item()
            test_predictions.extend(predicted.tolist())
            labels_test_total.extend(one_hot_targets.tolist())
            test_probabilities.extend(probs.tolist())

    test_loss /= len(dataloader)
    test_accuracy = 100 * correct_test / len(dataloader.dataset)

    print(f"Test Accuracy: {test_accuracy:.2f}%")
    print(f"Test Loss: {test_loss:.4f}")

    return torch.tensor(labels_test_total), torch.tensor(test_predictions), torch.tensor(test_probabilities), test_loss, test_accuracy
```

Evaluation and Visualization:

1. Compute the accuracy and print it
2. Print the classification report which includes Precision, Recall and F1 score for each class
3. Plot Loss and Accuracy curves to visualize the training progress
4. Plot the confusion matrix to display how well the model predicts for each class

Displaying Misclassified Images:

This function was made to identify the misclassified images. The process involves performing a forward pass through the test dataset where we identify the misclassified images by checking if the true label is not the same as the predicted label. We store these images along with their actual and predicted labels. The images are then plotted using matplotlib to get an idea of what the errors were like.

```
def display_misclassified_images(model, test_loader, device, class_names, num_images=5):

    model.eval()
    misclassified_images = []
    true_labels = []
    predicted_labels = []

    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            labels = F.one_hot(labels, 10).float()
            outputs = model(inputs)
            preds = outputs.argmax(1)

            for i in range(len(labels)):
                if preds[i] != labels[i]:
                    misclassified_images.append(inputs[i].cpu())
                    true_labels.append(labels[i].cpu().item())
                    predicted_labels.append(preds[i].cpu().item())

                if len(misclassified_images) >= num_images:
                    break
            if len(misclassified_images) >= num_images:
                break

    fig, axes = plt.subplots(1, num_images, figsize=(15, 5))

    for i in range(num_images):
        img = misclassified_images[i].permute(1, 2, 0).numpy()
        true_label = class_names[true_labels[i]]
        pred_label = class_names[predicted_labels[i]]

        axes[i].imshow(img)
        axes[i].set_title(f"True: {true_label}\nPred: {pred_label}", fontsize=10, color='red')
        axes[i].axis("off")

    plt.show()
```

Task:
Implement the architecture on a different dataset like the CIFAR-100 or MNIST and see how it performs. Doing this will make you understand the model better and will show how it can adapt to different scenarios. Also doing it on a bigger dataset would let you understand the limitations of the model.

Team Contribution:

Team Member	Video	Report	Code
Dharaneeshwar Shrisai Kumaraguru	50%	40%	60%
Kisore Senthilkumar	50%	60%	40%

References:

https://www.digitalocean.com/community/tutorials/capsule-networks

https://www.geeksforgeeks.org/capsule-neural-networks-ml/

https://www.kaggle.com/code/fizzbuzz/beginner-s-guide-to-capsule-networks

http://hdc.cs.arizona.edu/~mwli/understanding-capsule-network/writing/

https://teksands.ai/blog/introduction-to-capsule-networks

https://medium.com/analytics-vidhya/introduction-to-capsule-networks-66ebcdde4837

https://arxiv.org/abs/1710.09829